

TRABAJO PRACTICO

SUPER ELIZABETH SIS

Programacion 1- Primer Semestre 2026 -Turno Mañana

COMISION: 07

INTEGRANTES:

- Azul Carrizo
- Gabriel Moreira - moreira04gabriel@gmail.com - 45220898
- Luz romero – luzromt987@gmail.com - 47053997

PROFESORES:

- Nahuel Sauma
- Cesar Niveyro

Descripción del juego

El juego desarrollado es una adaptación del clásico “Mario bros” desarrollado en un entorno de “Islas” que se implementó como piso con ciertas dificultades y distintos largos para que el personaje “princesa Elizabeth” no caiga al vacío y así pueda llegar al castillo para lograr la victoria, este camino no debía ser fácil así que se implementaron enemigos voladores viniendo de ambos lados del entorno para dificultar la victoria de la princesa quitando vidas y en el peor caso provocando su muerte y la finalización del juego mostrando un cartel de derrota, estos enemigos cuentan con una sola vida la cual termina cuando impactan con la princesa o cuando salen del entorno en caso de no haber impacto, al momento de su muerte instantáneamente se crea otro enemigo para mantener siempre la misma cantidad y la misma dificultad en el juego. Para detener a estos enemigos la princesa Elizabeth cuenta con un disparo de un proyectil que se solo puede usar una vez hasta que el disparo salga del entorno o impacte con un enemigo, luego de cualquiera de esos dos casos la princesa puede volver a disparar. La princesa debe dirigirse hacia la derecha del entorno para poder avanzar ya que para la izquierda va a ser imposible en este entorno de “línea recta” hacia el castillo porque el mapa solo se va a mover cuando la Elizabeth se dirija hacia la derecha. Una vez el juego se da por finalizado se muestra un cartel de victoria y la princesa Elizabeth pudo cumplir su misión

CLASE PERSONAJE:

Esta clase representa al personaje el cual el usuario puede manejar en el juego.

La clase Personaje gestiona su posición, tamaño, movimiento, y detección de colisiones con islas mediante métodos que calculan sus bordes y un sistema de disparo orientado al puntero. El código define el movimiento, las colisiones por cuatro lados y el disparo inicial.

Variables:

int x: Coordenada horizontal del personaje en la pantalla.

int y: Coordenada vertical del personaje en la pantalla.

int ancho: Extensión horizontal es decir ancho del personaje.

int alto: Extensión vertical es decir el alto ocupa personaje.

Proyectil disparo: el proyectil disparado por el personaje. Si no hay proyectiles en movimiento, almacena null.

Constructor:

public Personaje(int x, int y, int ancho, int alto): Inicializa una nueva instancia definiendo su posición de nacimiento en el plano coordenado y sus dimensiones espaciales.

public void dibujar(Entorno e): Utiliza la librería gráfica incorporada para trazar un rectángulo de color rojo sólido (Color.RED) centrado en las coordenadas (x, y) actuales.

public void moverIzquierda(): Reduce la coordenada x en 5 unidades. Desplaza al personaje hacia la izquierda de la pantalla.

public void moverDerecha(): Incrementa la coordenada x en 5 unidades. Desplaza al personaje hacia la derecha de la pantalla.

public void moverArriba(): Reduce la coordenada y en 5 unidades. Eleva al personaje en pantalla (utilizado durante la acción de salto).

public void moverAbajo(): Incrementa la coordenada y en 5 unidades. Desplaza al personaje hacia el suelo (asociado a la acción del sistema de gravedad).

Métodos para combatir enemigos:

public void disparar(int mouseX, int mouseY)

Genera un disparo hacia el cursor del mouse utilizando para esto recibir las coordenadas x e y del mouse.

Calcula la distancia horizontal (deltaX) y vertical (deltaY) desde el centro del personaje hasta las coordenadas exactas de clic del ratón. Utiliza estos valores para instanciar un objeto de la clase Proyectil pasándole las magnitudes del vector de dirección para que la bala viaje en la trayectoria lineal apuntada.

Estos métodos calculan el perímetro cuadrado o caja de colisión del personaje dividiendo sus dimensiones simétricamente a partir de su punto de anclaje central (x, y):

public int bordeDerecho(): Retorna la coordenada X del lateral derecho del cuerpo ($x + \text{ancho} / 2$).

public int bordeIzquierdo(): Retorna la coordenada X del lateral izquierdo del cuerpo ($x - \text{ancho} / 2$).

public int bordeInferior(): Retorna la coordenada Y de la planta de los pies ($y + \text{alto} / 2$).

public int bordeSuperior(): Retorna la coordenada Y del extremo de la cabeza (y - alto / 2).

Métodos de Detección de Colisiones

Todos estos métodos recorren el arreglo completo de islas del mapa. Filtran los espacios vacíos (null) y también lógica de superposición de intervalos para frenar el movimiento antes de atravesar las islas :

public boolean colisionaPorIzquierda(Isla[] islas)

Comprueba si el borde izquierdo del héroe choca con el lateral derecho de alguna isla en un margen crítico de 5 píxeles, verificando al mismo tiempo que las alturas de ambos cuerpos se crucen verticalmente.

public boolean colisionaPorDerecha(Isla[] islas)

Comprueba si el borde derecho del héroe impacta el lateral izquierdo de una isla dentro del margen de aproximación de 5 píxeles, garantizando que el personaje no camine a través de paredes sólidas.

public boolean colisionaPorArriba(Isla[] islas)

Detecta si la parte superior del personaje choca contra la base inferior de alguna isla al saltar. Si da verdadero, frena el ascenso.

public boolean colisionaPorAbajo(Isla[] islas)

Determina si la parte inferior del personaje están apoyados sobre la superficie superior de una isla sólida. Sirve para desactivar la gravedad y estabilizar el estado del jugador en el piso firme.

Métodos Auxiliares, Getters y Setters

Getters y Setters: Encapsulan el acceso a los atributos privados (x, y, ancho, alto, disparo), permite que juego lea o modifique las coordenadas de el personaje es decir la princesa (por ejemplo, al caer al vacío o teletransportarse tras perder una vida).

Clase Proyectil:

La clase Proyectil representa el disparo lanzado por el personaje principal hacia la posición del cursor del mouse. Su propósito clave es calcular vectores de trayectoria en ángulos libres utilizando trigonometría, actualizar el desplazamiento de la bala y procesar las colisiones físicas contra los enemigos.

Atributos de la Clase

double x: Coordenada horizontal actual de la bala en la pantalla. con precisión decimal (double) para suavizar trayectorias en diagonal.

double y: Coordenada vertical actual de la bala en la pantalla (también con precisión decimal).

int diametro: Tamaño del cuerpo circular del proyectil en píxeles.

double deltaX: cuántos píxeles se mueve en X por fotograma

double deltaY: píxeles se mueve en Y.

double velocidad: Magnitud o rapidez escalar fija del proyectil (configurada en 8 píxeles por fotograma).

Constructor

public Proyectil(int x, int y, int diametro, double deltaX, double deltaY)

Fabrica un proyectil en las coordenadas de origen del personaje y calcula de forma inmediata su vector de dirección hacia el cursor.

Algoritmo Vectorial (Trigonometría):

Utiliza $(\text{Math.sqrt}(\text{deltaX} * \text{deltaX} + \text{deltaY} * \text{deltaY}))$ para obtener la **distancia total** (hipotenusa) entre el jugador y el clic del ratón.

Si la distancia es válida (distinta de cero), **normaliza** el vector dividiendo los deltas por la distancia (obteniendo un vector de longitud 1) y los multiplica por la variable velocidad. Esto garantiza que la bala viaje exactamente a 8 píxeles por fotograma, sin importar si el clic se hizo cerca o lejos del jugador.

Si la distancia da cero (clic sobre el personaje), el sistema predetermina un disparo recto hacia el lateral derecho de la pantalla por seguridad.

Métodos de Movimiento y Gráficos

public void mover()

Suma continuamente los valores dinámicos calculados de deltaX y deltaY a las posiciones actuales de x e y. Esto logra el avance lineal autónomo y fluido de la bala en cualquier dirección de la pantalla en cada *tick* del juego.

public void dibujar(Entorno e)

utiliza de la librería Entorno para trazar un círculo. Realiza un casteo de las coordenadas de double a int ((int)this.x, (int)this.y).

Métodos de Detección de Colisiones

public boolean colisionaConObstaculo(Enemigo enemigo)

Evalúa si el proyectil impactó físicamente a algún enemigo.

Primero filtra que el enemigo no sea un objeto nulo (null).

Utiliza la función de valor absoluto Math.abs() para calcular la distancia de separación entre el centro del proyectil y el centro del enemigo.

Si la distancia entre los ejes X es menor a la suma de sus mitades (radio de la bala + mitad del ancho del rival) **Y** la distancia en los ejes Y cumple el mismo criterio, el método determina una colisión exitosa retornando true.

Getters y Encapsulamiento

public int getX() y public int getY()

Retornan las coordenadas físicas actuales de la bala convertidas a números enteros (int). Son leídas en Juego.java para validar si el proyectil se escapó de los límites de la ventana (izquierda, derecha, arriba o abajo) y destruirlo (null).

Clase Enemigo:

En esta clase hemos usado las siguientes variables de instancia:

int x, y; estas variables representan la posición del enemigo en la pantalla.

int ancho, alto; estas variables determinan el tamaño del enemigo para dibujarlo y detectar colisiones.

`int velocidad`; esta variable indica la velocidad y dirección de movimiento del enemigo. Si el valor es positivo, el enemigo se mueve hacia la derecha, y si es negativo, se mueve hacia la izquierda.

En esta clase tenemos los siguientes métodos:

`dibujar`: este método representa gráficamente al enemigo en pantalla utilizando un círculo de color rojo centrado en sus coordenadas.

`mover`: este método actualiza la posición horizontal del enemigo sumando su velocidad a la coordenada x, permitiendo su desplazamiento automático.

`moverIzquierda`: este método desplaza manualmente al enemigo hacia la izquierda según el valor recibido por parámetro.

`bordeDerecho`, `bordeIzquierdo`, `bordeSuperior` y `bordeInferior`: estos métodos se utilizan para calcular los límites del enemigo y facilitar las detecciones de colisiones con otros objetos del juego.

`getters` y `setters`: estos métodos permiten obtener y modificar los valores de las variables de instancia del enemigo.

Clase Isla:

En esta clase hemos usado las siguientes variables de instancia:

`int x, y`; estas variables representan la posición de la isla en la pantalla.

`int ancho, alto`; estas variables determinan el tamaño de la isla utilizada como plataforma dentro del juego.

En esta clase tenemos los siguientes métodos:

dibujar: este método representa gráficamente la isla en pantalla utilizando un rectángulo de color verde.

moverIzquierda: este método desplaza la isla hacia la izquierda según el valor recibido por parámetro. Se utiliza para generar el efecto de movimiento del mapa o scroll de cámara.

bordeDerecho, bordeIzquierdo, bordeSuperior y bordeInferior: estos métodos calculan los límites de la isla y permiten detectar colisiones con el personaje u otros objetos del juego.

getters y setters: estos métodos permiten obtener y modificar los valores de las variables de instancia de la isla.

Clase Castillo:

En esta clase hemos usado las siguientes variables de instancia:

int x, y; estas variables representan la posición del castillo en la pantalla.

int ancho, alto; estas variables determinan el tamaño del castillo utilizado como objetivo final del juego.

En esta clase tenemos los siguientes métodos:

dibujar: este método representa gráficamente el castillo en pantalla utilizando un rectángulo de color azul.

moverIzquierda: este método desplaza el castillo hacia la izquierda según el valor recibido por parámetro. Se utiliza para acompañar el movimiento del mapa.

bordeDerecho, bordelzquierdo, bordeSuperior y bordeInferior: estos métodos calculan los límites del castillo y permiten detectar la colisión con el personaje para saber si se gana el juego.

getters y setters: estos métodos permiten obtener y modificar los valores de las variables de instancia del castillo.

CODIGO FUENTE IMPLEMENTADO EN JUEGO.JAVA

```
package juego;

import java.util.Random;

import entorno.Entorno;
import entorno.InterfaceJuego;

public class Juego extends InterfaceJuego {
    private Entorno entorno;
    private Personaje p;
    private Isla[] islas;
    private Enemigo[] enemigos;
    private int salto = 0;
    private Castillo castillo;
    private boolean gano;
    private int cantidadVidas;
    private boolean perdio;

    // CONSTRUCTOR JUEGO
    Juego() {
        this.entorno = new Entorno(this, "Proyecto para TP", 800, 600);
        gano = false;
        perdio = false;
        cantidadVidas = 5;

        // LLAMADAS de metods DE CREACIÓN
        crearPersonaje();
        crearIslas();
        crearEnemigos();

        castillo = new Castillo(3005, 300, 120, 200);
        this.entorno.iniciar();
    }
}
```

```
// CONSTRUCTOR JUEGO
Juego() {
    this.entorno = new Entorno(this, "Proyecto para TP", 800, 600);
    gano = false;
    perdio = false;
    cantidadVidas = 5;

    // LLAMADAS de metodos DE CREACIÓN
    crearPersonaje();
    crearIslas();
    crearEnemigos();

    castillo = new Castillo(3005, 300, 120, 200);
    this.entorno.iniciar();
}
}
```

```
// MÉTODO TICK PRINCIPAL
public void tick() {
    if (perdio) {
        entorno.escribirTexto("perdiste", 350, 300);
        return;
    }
    if (gano) {
        entorno.escribirTexto("GANASTE", 350, 300);
        return;
    }

    // VIDAS
    entorno.escribirTexto("Vidas: " + cantidadVidas, 20, 30);

    // Dibujado base
    if (p != null) p.dibujar(entorno);
    if (p != null && p.getDisparo() != null) p.getDisparo().dibujar(entorno);

    actualizarIslas();
    actualizarEnemigos();
}
```

```

// CONTROL DE MOVIMIENTO
if (p != null) {
    // Movimiento Izquierda
    if (entorno.estaPresionada(entorno.TECLA_IZQUIERDA) && p.getX() - p.getAncho() / 2 > 0) {
        if (!p.colisionaPorIzquierda(islas)) {
            p.moverIzquierda();
        }
    }

    // Movimiento Derecha
    if (entorno.estaPresionada(entorno.TECLA_DERECHA) && p.getX() + p.getAncho() / 2 < entorno.ancho()) {
        if (!p.colisionaPorDerecha(islas)) {
            if (p.getX() < 400 || esFinDelMapa()) {
                p.moverDerecha();
            } else {
                moverMundo(5);
            }
        }
    }
}

```

```

// Salto
if (entorno.estaPresionada(entorno.TECLA_ARRIBA) && p.colisionaPorAbajo(islas) && salto == 0) {
    salto = 30;
}

// Sistema de disparo
if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && p.getDisparo() == null) {
    p.disparar(entorno.mouseX(), entorno.mouseY());
}

if (p.getDisparo() != null) {
    p.getDisparo().mover();
    if (p.getDisparo().getX() < 0 || p.getDisparo().getY() < 0 || p.getDisparo().getX() > entorno.ancho() || p.getDisparo().getY() > entorno.alto()) {
        p.setDisparo(null);
    }
}

// Gravedad y saltos
if (!p.colisionaPorAbajo(islas)) {
    p.setY(p.getY() + 2);
}

if (salto > 0) {
    if (!p.colisionaPorArriba(islas)) {
        p.setY(p.getY() - 6);
        salto--;
    } else {
        salto = 0;
    }
}

```

```

// Gravedad y saltos
if (!p.colisionaPorAbajo(islas)) {
    p.setY(p.getY() + 2);
}

if (salto > 0) {
    if (!p.colisionaPorArriba(islas)) {
        p.setY(p.getY() - 6);
        salto--;
    } else {
        salto = 0;
    }
}

// Caída al vacío
if (p.getY() > entorno.alto()) {
    descontarVida();
}

// Colisión con la victoria
if (castillo != null && p.bordeDerecho() >= castillo.bordeIzquierdo() && p.bordeIzquierdo() <= castillo.bordeDerecho() && p.borde
    gana = true;
}

```

```

        // fin de juego
        if (cantidadVidas <= 0) {
            perdio = true;
        }

        for (int i = 0; i < enemigos.length; i++) {

            if (enemigos[i] == null) {
                enemigos[i] = generarEnemigo();
            }
        }
    }

    // MÉTODO: CREAR PERSONAJE
    private void crearPersonaje() {
        this.p = new Personaje(400, 300, 20, 50);
    }

```

```

// MÉTODO: CREAR ISLAS (Usa el Random de forma local adentro del método)
private void crearIslas() {
    this.islas = new Isla[49];

    Random r = new Random();
    for (int i = 0; i < islas.length; i++) {
        int columna = i / 3;
        int fila = i % 3;
        int x = 100 + (columna * 180);
        if (fila == 0) {
            islas[i] = new Isla(x, 500, 140, 200);
        } else if (fila == 1) {
            int y = 330 ;
            islas[i] = new Isla(x + r.nextInt(200), y, 100, 15);
        } else {
            int y = 220 ;
            islas[i] = new Isla(x + r.nextInt(150), y, 100, 15);
        }
    }
}

// MÉTODO: CREAR ENEMIGOS
private void crearEnemigos() {
    this.enemigos = new Enemigo[4];

    for (int i = 0; i < enemigos.length; i++) {
        enemigos[i] = generarEnemigo();
    }
}

```

```

// MÉTODO: CREAR ENEMIGOS
private void crearEnemigos() {
    this.enemigos = new Enemigo[4];

    for (int i = 0; i < enemigos.length; i++) {
        enemigos[i] = generarEnemigo();
    }
}

// MÉTODO: MOVER MUNDO
private void moverMundo(int velocidad) {
    for (int i = 0; i < islas.length; i++) {
        if (islas[i] != null) {
            islas[i].setX(islas[i].getX() - velocidad);
        }
    }
    if (castillo != null) {
        castillo.setX(castillo.getX() - velocidad);
    }

    for (int i = 0; i < enemigos.length; i++) {
        if (enemigos[i] != null) {
            enemigos[i].setX(enemigos[i].getX() - velocidad);
        }
    }
}

```

```

private boolean esFinDelMapa() {
    int xDeLaIslaMasLejana = 0;
    for (int i = 0; i < islas.length; i++) {
        if (islas[i] != null && islas[i].getX() > xDeLaIslaMasLejana) {
            xDeLaIslaMasLejana = islas[i].getX();
        }
    }
    return xDeLaIslaMasLejana <= 730;
}

private void actualizarIslas() {
    for (int i = 0; i < islas.length; i++) {
        if (islas[i] != null) {
            islas[i].dibujar(entorno);
        }
    }
    if (castillo != null) {
        castillo.dibujar(entorno);
    }
}

```

```
// Metodo: Para qd los Enemigos debe ser independiente
private void actualizarEnemigos() {
    for (int i = 0; i < enemigos.length; i++) {
        if (enemigos[i] == null) continue;

        enemigos[i].mover();
        enemigos[i].dibujar(entorno);

        if (enemigos[i].getX() < -50) {
            enemigos[i] = null;
            continue;
        } else if (enemigos[i].getX() > 850) {
            enemigos[i] = null;
            continue;
        }

        if (p != null && p.getDisparo() != null && p.getDisparo().colisionaConObstaculo(enemigos[i])) {
            enemigos[i] = null;
            p.setDisparo(null);
            continue;
        }

        if (p != null && p.bordeDerecho() >= enemigos[i].bordeIzquierdo() && p.bordeIzquierdo() <= enemigos[i].bordeDerecho() && p.bordeInferior() >= enemigos[i].bordeSuperior() && p.bordeSuperior() <= enemigos[i].bordeInferior()) {
            enemigos[i] = null;
            descontarVida();
        }
    }
}
```

```
private Enemigo generarEnemigo() {
    Random r = new Random();
    int velocidad = 2;

    // dos alturas fijas
    int yAlta = 120;
    int yBaja = 270;
    int yFinal;

    // una de las dos alturas
    int elegirY = r.nextInt(2); // Puede dar 0 o 1
    if (elegirY == 0) {
        yFinal = yAlta; // toca la altura 120
    } else {
        yFinal = yBaja; // toca la altura 270
    }

    int separacionXdeEnemigo = r.nextInt(200);
    // un boolean q Verdadero = Izquierda y Falso = Derecha
    if (r.nextBoolean()) {
        int x = -10 - separacionXdeEnemigo;
        return new Enemigo(x, yFinal, 30, 30, velocidad);
    } else {
        int x = 810 + separacionXdeEnemigo;
        return new Enemigo(x, yFinal, 30, 30, -velocidad);
    }
}
```

Conclusión:

En este trabajo práctico usamos GitHub. Fue un trabajo práctico colaborativo esta vez en Eclipse. En esta ocasión, al hacer el trabajo práctico, hubo muchos cambios y opiniones sobre qué hacer o qué lógica implementar, comentadas entre los integrantes. La comunicación durante este trabajo práctico fue mediante un grupo de WhatsApp.

En cuanto a inconvenientes, el principal nuestro fue al crear las islas y hacer el movimiento hacia la izquierda del mundo. Por otro lado, respecto a la división de tareas, tanto las clases como los ítems de funcionalidades obligatorias a cumplir se dividieron al inicio, aunque donde hubo problemas tuvimos que resolverlos en colaboración grupal.

En resumen, y para finalizar esta reflexión, podemos decir que en el proceso de este trabajo práctico la enseñanza principal fue la resolución de problemas y el trabajo grupal, ya que revisamos entre todos las líneas de código de cada integrante para cumplir con los requerimientos de los ítems y corregir errores o código que estaba de más. De este modo nos aseguramos de que finalmente estuviera todo funcionando bien y cumpliendo cada requerimiento solicitado en el enunciado del trabajo práctico.